

JavaScript Graphics: An Overview

JavaScript Graphics allow developers to create dynamic, interactive, and visually appealing content directly in the web browser. From simple shapes to complex 3D environments, JavaScript provides the tools necessary to render graphics programmatically.

Before advanced APIs, web graphics were limited to static images (GIF, JPEG, PNG). Today, JS Graphics power:

- **Data Visualization:** Charts, maps, and dashboards.
- **Web Gaming:** Real-time 2D and 3D rendering.
- **UI Enhancements:** Transition effects and custom icons.

Definition: JavaScript Graphics refers to the programmatic creation and manipulation of visual elements on a web page using APIs like Canvas, SVG, and WebGL.

1. The HTML5 Canvas API

The `<canvas>` element is used to draw graphics via JavaScript. It is "pixel-based" (raster), meaning it draws pixels on a fixed-size grid.

Basic Implementation

```
<canvas id="myCanvas" width="200" height="100"></canvas>

<script>
  const c = document.getElementById("myCanvas");
  const ctx = c.getContext("2d");
  ctx.fillStyle = "#FF0000";
  ctx.fillRect(20, 20, 150, 75);
</script>
```

The `getContext("2d")` method returns an object that provides methods and properties for drawing and manipulating images and graphics on a canvas.

Drawing on Canvas

The Canvas API provides several methods for drawing shapes and paths:

- `fillRect(x,y,w,h)` : Draws a filled rectangle.
- `strokeRect(x,y,w,h)` : Draws a rectangle outline.
- `clearRect(x,y,w,h)` : Clears a rectangular area.

Drawing Paths (Lines & Circles)

```
ctx.beginPath();
ctx.moveTo(0, 0); // Start point
ctx.lineTo(200, 100); // End point
ctx.stroke(); // Draw the line

// Drawing a Circle
ctx.beginPath();
ctx.arc(95, 50, 40, 0, 2 * Math.PI);
ctx.stroke();
```

2. SVG (Scalable Vector Graphics)

SVG is an XML-based format for vector graphics. Unlike Canvas, SVG is object-based, meaning every drawn shape is part of the Document Object Model (DOM).

Key Features of SVG:

- **Scalability:** SVG images never lose quality, no matter how much they are zoomed or resized.
- **DOM Accessibility:** You can attach event listeners (like click) to individual SVG shapes.
- **Searchability:** Since it is XML, the text inside SVGs is searchable by search engines.

```
<svg width="100" height="100">  
  <circle cx="50" cy="50" r="40" stroke="black" stroke-width="3"  
  fill="red" />  
</svg>
```

Choosing the Right Tool: Canvas vs. SVG

Feature	Canvas	SVG
Rendering Agent	Browser	Browser
Dependency	Resolution Dependent	Resolution Independent
Performance	Better for high-density small area	Better for large area interaction
Event Handling	No (Single pixel surface)	Yes (Element based)
Best Use	Games, Pixels, Heavy JS	High fidelity icons, Diagrams
Complexity	Faster for complex animations	Slows down with too many nodes

3. CSS + JavaScript Graphics

JavaScript can be used to control CSS properties dynamically to create smooth graphical effects without using Canvas or SVG.

Dynamic Style Manipulation

```
const elem = document.getElementById("box");  
elem.style.transform = "rotate(45deg)";  
elem.style.borderRadius = "50%";  
elem.style.boxShadow = "10px 10px 5px grey";
```

JS-Controlled Animations

Using `requestAnimationFrame()`, JS can create high-performance animations by changing CSS properties on every frame refresh.

Manipulating Images

JavaScript allows you to draw external images onto a canvas, which is the basis for most web-based games and photo editors.

```
const img = new Image();
img.onload = function() {
  ctx.drawImage(img, 10, 10);
};
img.src = "picture.jpg";
```

Advanced Techniques

- **Cropping:** Drawing only a piece of an image.
- **Pixel Manipulation:** Using `getImageData()` to get an array of every pixel's color, allowing for custom filters like grayscale or sepia.

Drawing Text

Both Canvas and SVG allow for text rendering, though with different benefits.

Canvas Text

```
ctx.font = "30px Arial";  
ctx.fillText("Hello World", 10, 50);  
ctx.strokeText("Hollow Text", 10, 100);
```

SVG Text

```
<svg height="30" width="200">  
  <text x="0" y="15" fill="red">I love SVG!</text>  
</svg>
```

SVG text remains selectable and accessible to screen readers, whereas Canvas text is just "painted" pixels.

Visualizing Audio and Gradients

Graphics are often paired with other APIs. A common use case is a sound visualizer using the Web Audio API to draw bars on a Canvas.

Gradients

Instead of solid colors, you can use Linear or Radial gradients to transition between multiple colors.

```
// Create gradient
const grd = ctx.createLinearGradient(0, 0, 200, 0);
grd.addColorStop(0, "red");
grd.addColorStop(1, "white");

// Fill with gradient
ctx.fillStyle = grd;
ctx.fillRect(10, 10, 150, 80);
```

Introduction to WebGL

WebGL (Web Graphics Library) is a JavaScript API for rendering high-performance interactive 3D and 2D graphics within any compatible web browser without the use of plug-ins.

- **Hardware Acceleration:** WebGL uses the computer's GPU (Graphics Processing Unit) to render 3D scenes.
- **Complexity:** It is significantly more difficult than the 2D Canvas API.
- **Libraries:** Most developers use libraries like **Three.js** or **Babylon.js** to work with WebGL simplified.

CODEHIVE

Popular JavaScript Graphics Libraries

While native APIs are powerful, libraries provide high-level abstractions for common tasks.

- **Chart.js / D3.js:** The industry standard for creating professional-grade charts and data visualizations.
- **Three.js:** The most popular library for 3D on the web.
- **PixiJS:** A lighting-fast 2D engine often used for high-performance games.
- **Snap.svg:** Makes working with SVG as easy as jQuery makes working with the DOM.

Pro Tip: Don't reinvent the wheel. If you need a Bar Chart, use a library rather than drawing lines manually on a Canvas.

CODEHIVE

★ Exam Summary Points

- **JS Graphics:** Used to draw shapes, images, and animations programmatically.
- **Canvas API:** Pixel-based, resolution-dependent, fast for games and dynamic scenes.
- **SVG:** Vector-based, resolution-independent (high quality at any size), accessible via DOM.
- **CSS + JS:** Best for simple UI transforms, rotations, and standard button animations.
- **Applications:** Vital for data visualization (charts), gaming, and modern interactive UI design.
- **Context:** Remember `getContext( "2d" )` is the entry point for Canvas drawing.

Future of Web Graphics

With the rise of WebGPU, web-based graphics are becoming indistinguishable from native desktop applications, allowing for cinematic-quality visuals in the browser.